

SAE 2.01

Tutoriel #1

Mettre en place une application Flask

Création du projet, connexion à la base de données SAE2.04, première page.

BUT Informatique · IUT de Créteil-Vitry · Département Informatique

Ce premier tutoriel pose les fondations de l'application web. À l'issue de cette étape, vous disposerez d'une application Flask fonctionnelle, connectée à la base MySQL construite en SAE2.04, et capable d'afficher une première page exploitant les données de la base.

Prérequis

Disposer de votre base SAE2.04 (sae204_XX_bd) remplie avec les 9 tables de dimensions. Si elle ne l'est pas, l'enseignant fournit un fichier sae204_ideal.sql à importer (voir section 2.4). Disposer de Python 3.10 ou supérieur et d'un éditeur type VSCode.

1. Flask et architecture MVC

1.1 Qu'est-ce que Flask ?

Flask est un framework web Python minimaliste. Contrairement à d'autres outils (Django, Symfony...), il impose peu de contraintes : vous choisissez les bibliothèques additionnelles selon vos besoins. C'est un excellent terrain pour découvrir le développement web sans surcharge.

Une application Flask repose sur trois éléments essentiels :

- Des routes : des fonctions Python associées à des URL (par exemple /, /effectifs).
- Des templates HTML : des fichiers d'affichage, enrichis par le moteur Jinja2.
- Un serveur de développement intégré qui rend l'application accessible sur localhost.

1.2 L'architecture MVC

Pour garder le code lisible et évolutif, l'application sera organisée selon le patron MVC (Modèle – Vue – Contrôleur). Ce découpage sépare les responsabilités :

Rôle	Dans notre projet
------	-------------------

Modèle (M)	Les données : classes ORM (Region, Departement...) et services métier (AmeliAPI).
Vue (V)	L'affichage : templates HTML (Jinja2), CSS, JavaScript.
Contrôleur (C)	Le chef d'orchestre : routes Flask qui reçoivent la requête et rendent une vue.

2. Mise en place du projet

2.1 Installation des dépendances

Créer un nouveau dossier pour le projet, puis installer les bibliothèques nécessaires :

```
pip install flask sqlalchemy pymysql python-dotenv requests
```

Bibliothèque	Rôle
flask	Framework web
sqlalchemy	ORM (déjà utilisé en SAE2.04)
pymysql	Driver MySQL
python-dotenv	Lecture des variables .env
requests	Appels à l'API ameli.fr (tutoriel #2)

2.2 Structure du projet

Créer la structure de dossiers suivante (les dossiers vides seront remplis au fil des tutoriels) :

```
SAE201-app/
├── app.py           ← point d'entrée
├── config.py        ← configuration
├── .env             ← identifiants (NE PAS versionner)
├── .env.example     ← modèle de .env
├── .gitignore
├── models/          ← classes ORM + moteur BDD
│   ├── __init__.py
│   ├── db.py
│   └── dimensions.py
├── services/        ← services métier (tuto #2)
│   └── __init__.py
├── controllers/     ← routes Flask
│   └── __init__.py
```

```

├── accueil.py
├── templates/           ← fichiers HTML Jinja2
│   ├── base.html
│   └── accueil.html
├── static/              ← CSS, JS, images
│   ├── css/
│   └── style.css

```

À propos des fichiers `__init__.py`

Ces fichiers (même vides) indiquent à Python que le dossier est un « package », ce qui permet de faire des imports relatifs (par exemple `from models.dimensions import Region`).

2.3 Fichier `.env`

Le fichier `.env` contient les informations sensibles (identifiants, mots de passe). Il ne doit jamais être versionné dans Git. Créer ce fichier à la racine avec vos identifiants de la SAE2.04 :

Fichier `.env`

```

DB_USER      = sae204_XX_user
DB_PASSWORD  = *****
DB_HOST      = mysql-sae204.alwaysdata.net
DB_NAME      = sae204_XX_bd
FLASK_ENV    = development
SECRET_KEY   = changez-moi-par-une-chaine-aleatoire

```

Créer également un fichier `.env.example` (celui-ci sera versionné) avec les clés mais SANS les valeurs :

Fichier `.env.example`

```

DB_USER      =
DB_PASSWORD  =
DB_HOST      =
DB_NAME      =
FLASK_ENV    = development
SECRET_KEY   =

```

Si votre équipe utilise Git (optionnel)

L'utilisation d'un dépôt Git (GitHub, GitLab...) n'est pas imposée mais recommandée pour partager le code à plusieurs. Si vous choisissez Git, créer aussi un fichier `.gitignore` à la racine, qui indique les fichiers à exclure du suivi :

Fichier `.gitignore`

```

.env
__pycache__/
*.pyc

```

```
venv/  
.vscode/
```

Si vous préférez ne pas utiliser Git pour l'instant, ignorer cette étape : l'application fonctionnera de la même manière. Le partage entre coéquipiers peut alors se faire via un dossier Drive/OneDrive ou par envoi d'archive ZIP.

2.4 Importer la base de données fournie

Votre application a besoin des 9 tables de dimensions de la SAE2.04 (régions, départements, professions...). Plutôt que de les reconstruire, l'enseignant vous fournit un fichier `sae204_ideal.sql` : une base déjà remplie, prête à l'emploi. Il suffit de l'importer dans VOTRE base `sae204_XX_bd`.

Depuis phpMyAdmin (sur Alwaysdata, ou en local) :

1. Sélectionner d'abord VOTRE base `sae204_XX_bd` dans la colonne de gauche.
2. Ouvrir l'onglet « Importer ».
3. Choisir le fichier `sae204_ideal.sql`, puis cliquer sur « Exécuter ».
4. Vérifier que les 9 tables apparaissent à gauche, avec leurs données.

Points importants

Bien sélectionner VOTRE base AVANT d'importer : le fichier ne crée pas de base, il remplit celle qui est sélectionnée. Le fichier contient des instructions `DROP TABLE` : il peut être réimporté sans erreur si besoin. Cette base est directement compatible avec le code de l'application (mêmes noms de tables et de colonnes que les classes ORM du dossier `models/`).

3. Configuration et connexion à la base

3.1 Fichier config.py

Centraliser la configuration dans une classe Config rend l'application plus propre : on charge les variables d'environnement une fois, et on y accède via des attributs de classe.

config.py

```
import os
from dotenv import load_dotenv

load_dotenv()

class Config:
    """Configuration de l'application."""
    DB_USER      = os.getenv("DB_USER")
    DB_PASSWORD   = os.getenv("DB_PASSWORD")
    DB_HOST      = os.getenv("DB_HOST")
    DB_NAME      = os.getenv("DB_NAME")
    SECRET_KEY    = os.getenv("SECRET_KEY", "dev-secret")

    @classmethod
    def db_url(cls):
        """Construit l'URL de connexion MySQL."""
        return (f"mysql+pymysql://{cls.DB_USER}:{cls.DB_PASSWORD}"
                f"@{cls.DB_HOST}/{cls.DB_NAME}")
```

POO en action

Ici, Config est une classe dont tous les attributs sont des variables de classe. La méthode db_url() est déclarée @classmethod : elle peut être appelée directement sur la classe sans créer d'instance. C'est l'application du principe d'encapsulation : la construction de l'URL est cachée, et si elle change un jour, un seul endroit est à modifier.

3.2 Moteur SQLAlchemy (models/db.py)

Ce fichier centralise l'accès à la base : création du moteur SQLAlchemy et fabrique de sessions.

models/db.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from config import Config

# Un seul moteur pour toute l'application
engine = create_engine(Config.db_url(), pool_recycle=3600)

# Fabrique de sessions ; chaque requête HTTP utilisera sa propre session
Session = sessionmaker(bind=engine)
```

Pourquoi pool_recycle=3600 ?

MySQL ferme les connexions inactives au bout d'un certain temps. Ce paramètre force SQLAlchemy à recycler les connexions toutes les heures, ce qui évite les erreurs « MySQL server has gone away ».

3.3 Modèles ORM (models/dimensions.py)

Copier ici le fichier models_dimensions.py de la SAE2.04. Les 9 classes (Region, Departement, ProfessionSante, etc.) sont réutilisées telles quelles.

Astuce : ajouter une méthode to_dict()

Pour faciliter les futures routes AJAX (tutoriel #2), ajoutez à chaque classe une méthode to_dict() qui renvoie un dictionnaire représentant l'objet. C'est une bonne illustration du principe d'encapsulation : l'objet sait lui-même se représenter.

4. Point d'entrée et première page

4.1 Fichier app.py

Le fichier app.py est le point d'entrée de l'application. Il crée l'instance Flask, enregistre les routes, puis lance le serveur de développement.

app.py

```
from flask import Flask
from config import Config
from controllers.accueil import bp_accueil

app = Flask(__name__)
app.config.from_object(Config)

# Enregistrement des contrôleurs (blueprints)
app.register_blueprint(bp_accueil)

if __name__ == "__main__":
    app.run(debug=True)
```

Les blueprints Flask

Un blueprint est un regroupement de routes. Il permet de découper l'application en modules thématiques (un blueprint par grande fonctionnalité). Nous en utilisons un par fichier dans le dossier controllers/.

4.2 Template de base (templates/base.html)

Tous les templates de l'application hériteront de ce template. Il définit la structure commune : en-tête, menu, pied de page. Les pages filles ne redéfinissent que la partie centrale.

templates/base.html

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>{% block titre %}Données de santé{% endblock %}</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css')
}}">
</head>
<body>
  <header>
    <h1>Données de santé libérale</h1>
    <nav>
      <a href="{{ url_for('accueil.index') }}">Accueil</a>
    </nav>
  </header>

  <main>
    {% block contenu %}{% endblock %}
  </main>

  <footer>
    <p>SAE2.01 • BUT Informatique • IUT Créteil-Vitry</p>
  </footer>
</body>
</html>
```

4.3 Contrôleur accueil

Le contrôleur définit une route / qui rendra la page d'accueil. Il récupère la liste des régions et des professions depuis la base pour les passer au template.

controllers/accueil.py

```
from flask import Blueprint, render_template
from models.db import Session
from models.dimensions import Region, ProfessionSante

bp_accueil = Blueprint("accueil", __name__)

@bp_accueil.route("/")
def index():
    """Page d'accueil : affiche les régions et professions."""
    session = Session()
    try:
        regions = session.query(Region).order_by(Region.libelle).all()
        professions = (session.query(ProfessionSante)
```

```

        .order_by(ProfessionSante.libelle).all())
    return render_template("accueil.html",
                           regions=regions,
                           professions=professions)

finally:
    session.close()

```

Pourquoi try / finally ?

Une session SQLAlchemy doit toujours être fermée, sinon la connexion reste ouverte. Le bloc try / finally garantit que session.close() est appelée même en cas d'erreur.

4.4 Template d'accueil (templates/accueil.html)

templates/accueil.html

```

{% extends "base.html" %}

{% block titre %}Accueil{% endblock %}

{% block contenu %}
    <h2>Bienvenue !</h2>
    <p>La base contient {{ regions|length }} régions et {{ professions|length }}
    professions.</p>

    <h3>Régions</h3>
    <ul>
        {% for region in regions %}
            <li>{{ region.code }} - {{ region.libelle }}</li>
        {% endfor %}
    </ul>

    <h3>Professions</h3>
    <ul>
        {% for prof in professions %}
            <li>{{ prof.libelle }}</li>
        {% endfor %}
    </ul>
{% endblock %}

```

Jinja2 : la syntaxe en deux balises

{{ ... }} affiche une valeur. {% ... %} exécute une instruction (for, if, extends, block). L'héritage via {% extends %} est très puissant : on écrit une seule fois la structure commune, et chaque page ne décrit que sa spécificité.

5. Premier lancement

Dans un terminal, à la racine du projet :

```
python app.py
```

Flask démarre et affiche une URL du type `http://127.0.0.1:5000`. Ouvrir cette adresse dans un navigateur : la page d'accueil doit apparaître avec la liste des régions et des professions.

Mode debug : recharge automatique

Grâce au paramètre `debug=True`, Flask relance automatiquement l'application à chaque modification d'un fichier Python. Les templates HTML sont eux aussi rechargés à chaque requête : pas besoin de redémarrer quand on modifie une vue.

5.1 Vérifications à faire

- La page affiche la liste des régions.
- La page affiche la liste des professions.
- Les listes sont triées par ordre alphabétique.
- Aucune erreur dans le terminal Flask.

Contrôle exact si vous avez importé `sae204_ideal.sql`

Avec la base fournie par l'enseignant (`sae204_ideal.sql`), vous devez voir EXACTEMENT 19 régions et 38 professions. Si ces nombres diffèrent, l'import s'est mal passé : recommencez la section 2.4.

5.2 Erreurs courantes

Erreur	Piste de résolution
Access denied for user	Vérifier <code>DB_USER</code> et <code>DB_PASSWORD</code> dans <code>.env</code> .
Can't connect to MySQL server	Vérifier <code>DB_HOST</code> . Tester la connexion avec DBeaver au préalable.
ModuleNotFoundError	Vérifier que pip install a bien été fait dans le bon environnement.
TemplateNotFound	Vérifier que les templates sont bien dans le dossier <code>templates/</code> à la racine.

6. Un peu de style avec CSS

Pour rendre la page plus agréable, ajouter un minimum de style dans le fichier `static/css/style.css`. Le template de base l'inclut déjà via `url_for`.

static/css/style.css

```
body {
  font-family: "Segoe UI", sans-serif;
  margin: 0;
  color: #333;
}

header {
  background: #2E74B5;
  color: white;
  padding: 1rem 2rem;
}

header nav a {
  color: white;
  margin-right: 1rem;
  text-decoration: none;
}

main {
  padding: 2rem;
  max-width: 900px;
  margin: auto;
}

footer {
  text-align: center;
  padding: 1rem;
  color: #777;
  border-top: 1px solid #ddd;
}
```

Bilan du tutoriel #1

À ce stade, votre application :

- se lance avec `python app.py` et est accessible sur `localhost:5000`,
- se connecte à la base SAE2.04 via SQLAlchemy,
- affiche une page d'accueil stylée avec les régions et les professions,
- respecte l'architecture MVC (models, controllers, templates séparés).

Le tutoriel #2 va introduire un formulaire de sélection, l'appel à l'API `ameli.fr`, et les premières visualisations graphiques.